



NumPy Complete Guide

This notebook covers NumPy fundamentals, array creation, attributes, operations, mathematical and statistical functions, indexing and slicing, array manipulation, utility functions, and a small case study using a structured dataset.

1. Introduction to NumPy

NumPy (Numerical Python) is a library used for working with numerical data in Python. Analysts use it because it provides fast, memory-efficient arrays and built-in mathematical operations that are much faster than plain Python lists.

Key reasons analysts use NumPy:

- Arrays store data of a single type, which makes them compact and fast.
- Operations are vectorized, meaning they run on entire arrays without writing loops.
- It is the foundation for other data analysis libraries like Pandas.

```
import numpy as np
print(np.__version__)
```

2.4.4

Arrays vs Python Lists

A Python list can hold mixed data types and each element is a separate object in memory, which adds overhead. A NumPy array holds elements of the same type in a continuous block of memory, which reduces overhead and enables fast computation.

```
py_list = [1, 2, 3, 4, 5]
np_array = np.array([1, 2, 3, 4, 5])

print("List:", py_list, type(py_list))
print("Array:", np_array, type(np_array))

List: [1, 2, 3, 4, 5] <class 'list'>
Array: [1 2 3 4 5] <class 'numpy.ndarray'>
```

Memory Efficiency

NumPy arrays use less memory than Python lists because they store elements in a fixed-size, homogeneous block rather than as individual Python objects.

```
import sys

py_list = list(range(1000))
np_array = np.arange(1000)

list_size = sys.getsizeof(py_list) + sum(sys.getsizeof(i) for i in py_list)
array_size = np_array.nbytes

print("Python list approx size (bytes):", list_size)
print("NumPy array size (bytes):", array_size)

Python list approx size (bytes): 36056
NumPy array size (bytes): 8000
```

Vectorization

Vectorization means applying an operation to an entire array at once instead of looping element by element. This is faster because NumPy performs the loop internally in optimized C code.

```
# Non-vectorized (Python loop)
py_list = list(range(1000000))
result = [x * 2 for x in py_list]
```

```
# Vectorized (NumPy)
np_array = np.arange(1000000)
result_np = np_array * 2

print(result_np[:5])
```

[0 2 4 6 8]

▼ Demo: Compare List vs Array Performance

```
import time

size = 1000000
py_list = list(range(size))
np_array = np.arange(size)

start = time.time()
py_result = [x * 2 for x in py_list]
py_time = time.time() - start

start = time.time()
np_result = np_array * 2
np_time = time.time() - start

print("Python list time:", py_time)
print("NumPy array time:", np_time)
print("NumPy is faster by approx:", py_time / np_time, "times")
```

Python list time: 0.07562947273254395
 NumPy array time: 0.003487825393676758
 NumPy is faster by approx: 21.68384715291544 times

▼ 2. Creating Arrays

NumPy provides several functions to create arrays with specific values, ranges, or random data.

▼ np.array()

Creates an array from a Python list or tuple.

```
a = np.array([1, 2, 3])
b = np.array([[1, 2], [3, 4]])
print(a)
print(b)
```

[1 2 3]
 [[1 2]
 [3 4]]

▼ np.arange()

Creates an array with evenly spaced values within a given range, similar to Python's range() function.

```
print(np.arange(0, 10, 2))
```

[0 2 4 6 8]

▼ np.linspace()

Creates an array of evenly spaced numbers over a specified interval, given the number of points instead of the step size.

```
print(np.linspace(0, 1, 5))
```

[0. 0.25 0.5 0.75 1.]

▼ np.logspace()

Creates an array of numbers evenly spaced on a logarithmic scale.

```
print(np.logspace(0, 3, 4))

[  1.   10.  100. 1000.]
```

▼ np.zeros(), np.ones(), np.empty()

- np.zeros() creates an array filled with zeros.
- np.ones() creates an array filled with ones.
- np.empty() creates an array without initializing values (contains garbage values, faster to create).

```
print(np.zeros((2, 3)))
print(np.ones((2, 3)))
print(np.empty((2, 2)))

[[0. 0. 0.]
 [0. 0. 0.]]
[[1. 1. 1.]
 [1. 1. 1.]]
[[  1.   10.]
 [ 100. 1000.]]
```

▼ np.eye(), np.identity()

Both create identity matrices (square matrices with 1s on the diagonal and 0s elsewhere). np.eye() is more flexible and allows rectangular matrices and diagonal offsets.

```
print(np.eye(3))
print(np.identity(3))

[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

▼ Random Arrays

NumPy's random module generates arrays with random values.

- np.random.rand() gives uniform random values between 0 and 1.
- np.random.randn() gives values from a standard normal distribution.
- np.random.randint() gives random integers within a range.
- np.random.choice() picks random elements from a given array.
- np.random.seed() fixes the random number generator so results are reproducible.

```
np.random.seed(42)

print(np.random.rand(3))
print(np.random.randn(3))
print(np.random.randint(1, 10, size=5))
print(np.random.choice([10, 20, 30, 40], size=3))

[0.37454012 0.95071431 0.73199394]
[-1.11188012 0.31890218 0.27904129]
[8 3 6 5 2]
[40 40 20]
```

▼ 3. Array Attributes & Properties

NumPy arrays have attributes that describe their structure and memory usage.

- ndim: number of dimensions
- shape: size of the array along each dimension

- size: total number of elements
- dtype: data type of the elements
- itemsize: size in bytes of each element
- nbytes: total size in bytes of the array

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

print("ndim:", arr.ndim)
print("shape:", arr.shape)
print("size:", arr.size)
print("dtype:", arr.dtype)
print("itemsize:", arr.itemsize)
print("nbytes:", arr.nbytes)
```

```
ndim: 2
shape: (2, 3)
size: 6
dtype: int64
itemsize: 8
nbytes: 48
```

✓ Reshaping

- reshape() changes the shape of an array without changing its data.
- ravel() returns a flattened array as a view (shares memory when possible).
- flatten() returns a flattened array as a copy.
- resize() changes the shape of an array in place.

```
arr = np.arange(12)

reshaped = arr.reshape(3, 4)
print(reshaped)

print(reshaped.ravel())
print(reshaped.flatten())

arr2 = np.arange(6)
arr2.resize(2, 3)
print(arr2)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[[0 1 2]
 [3 4 5]]
```

✓ Transpose

Transpose flips the axes of an array, turning rows into columns and vice versa. It can be done with the .T attribute or np.transpose().

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.T)
print(np.transpose(arr))
```

```
[[1 4]
 [2 5]
 [3 6]]
[[1 4]
 [2 5]
 [3 6]]
```

✓ 4. Array Operations

NumPy supports element-wise arithmetic, comparison, and logical operations, as well as broadcasting between arrays of different shapes.

Arithmetic Operations

`+`, `-`, `*`, `/`, `//`, `**`, `%` perform element-wise addition, subtraction, multiplication, division, floor division, exponentiation, and modulus.

```
a = np.array([10, 20, 30])
b = np.array([2, 4, 6])
```

```
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a ** 2)
print(a % b)
```

```
[12 24 36]
[ 8 16 24]
[ 20 80 180]
[5. 5. 5.]
[5 5 5]
[100 400 900]
[0 0 0]
```

Comparison Operators

`==`, `!=`, `>`, `<`, `>=`, `<=` compare elements and return a boolean array.

```
a = np.array([1, 2, 3, 4])
b = np.array([2, 2, 2, 2])
```

```
print(a == b)
print(a != b)
print(a > b)
print(a < b)
print(a >= b)
print(a <= b)
```

```
[False  True False False]
[ True False  True  True]
[False False  True  True]
[ True False False False]
[False  True  True  True]
[ True  True False False]
```

Logical Operations

- `np.logical_and()` returns True where both conditions are True.
- `np.logical_or()` returns True where at least one condition is True.
- `np.logical_not()` inverts a boolean array.

```
a = np.array([True, False, True])
b = np.array([True, True, False])
```

```
print(np.logical_and(a, b))
print(np.logical_or(a, b))
print(np.logical_not(a))
```

```
[ True False False]
[ True  True  True]
[False  True False]
```

Broadcasting

Broadcasting allows NumPy to perform operations on arrays of different shapes by automatically expanding the smaller array to match the larger one's shape, without copying data.

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
scalar = 10
vector = np.array([1, 0, 1])
```

```
print(arr + scalar)
print(arr + vector)
```

```
[[11 12 13]
 [14 15 16]]
[[2 2 4]
 [5 5 7]]
```

▼ Element-wise vs Aggregate Operations

Element-wise operations act on each element individually and return an array of the same shape. Aggregate operations combine elements into a single summary value, such as sum or mean.

```
arr = np.array([1, 2, 3, 4])

print("Element-wise square:", arr ** 2)
print("Aggregate sum:", arr.sum())
print("Aggregate mean:", arr.mean())
```

```
Element-wise square: [ 1  4  9 16]
Aggregate sum: 10
Aggregate mean: 2.5
```

▼ 5. Mathematical & Statistical Functions

▼ Basic Math Functions

`np.add()`, `np.subtract()`, `np.multiply()`, `np.divide()`, `np.power()`, `np.mod()` perform the same element-wise operations as the arithmetic operators but as explicit functions. `np.sqrt()` computes square root, `np.exp()` computes the exponential, `np.log()` computes natural log, `np.log10()` computes base-10 log.

```
a = np.array([4, 9, 16])
b = np.array([2, 3, 4])
```

```
print(np.add(a, b))
print(np.subtract(a, b))
print(np.multiply(a, b))
print(np.divide(a, b))
print(np.power(a, 2))
print(np.mod(a, b))
print(np.sqrt(a))
print(np.exp(b))
print(np.log(a))
print(np.log10(a))
```

```
[ 6 12 20]
[ 2  6 12]
[ 8 27 64]
[2.  3.  4.]
[ 16  81 256]
[0  0  0]
[2.  3.  4.]
[ 7.3890561  20.08553692 54.59815003]
[1.38629436 2.19722458 2.77258872]
[0.60205999 0.95424251 1.20411998]
```

▼ Trigonometric Functions

`np.sin()`, `np.cos()`, `np.tan()` compute trig ratios in radians. `np.arcsin()` computes the inverse sine. `np.degrees()` converts radians to degrees, and `np.radians()` converts degrees to radians.

```
angles = np.array([0, np.pi/2, np.pi])

print(np.sin(angles))
print(np.cos(angles))
print(np.tan(angles))
print(np.arcsin(np.array([0, 1])))
print(np.degrees(np.pi))
print(np.radians(180))
```

```
[0.0000000e+00 1.0000000e+00 1.2246468e-16]
[ 1.0000000e+00  6.123234e-17 -1.0000000e+00]
[ 0.0000000e+00  1.63312394e+16 -1.2246468e-16]
[0.          1.57079633]
180.0
3.141592653589793
```

✓ Rounding Functions

`np.round()` rounds to a given number of decimals. `np.floor()` rounds down to the nearest integer. `np.ceil()` rounds up to the nearest integer.

```
arr = np.array([1.234, 2.567, 3.891])

print(np.round(arr, 1))
print(np.floor(arr))
print(np.ceil(arr))
```

```
[1.2 2.6 3.9]
[1.  2.  3.]
[2.  3.  4.]
```

✓ Statistical Functions

`np.mean()` computes the average. `np.median()` computes the middle value. `np.std()` computes standard deviation. `np.var()` computes variance. `np.min()/np.max()` find the smallest/largest values. `np.sum()` adds all elements. `np.cumsum()` gives a running cumulative sum. `np.prod()` multiplies all elements. `np.percentile()` finds the value below which a given percentage of data falls.

```
arr = np.array([1, 2, 3, 4, 5, 6])

print("mean:", np.mean(arr))
print("median:", np.median(arr))
print("std:", np.std(arr))
print("var:", np.var(arr))
print("min:", np.min(arr))
print("max:", np.max(arr))
print("sum:", np.sum(arr))
print("cumsum:", np.cumsum(arr))
print("prod:", np.prod(arr))
print("percentile 50:", np.percentile(arr, 50))
```

```
mean: 3.5
median: 3.5
std: 1.707825127659933
var: 2.9166666666666665
min: 1
max: 6
sum: 21
cumsum: [ 1  3  6 10 15 21]
prod: 720
percentile 50: 3.5
```

✓ 6. Indexing & Slicing

✓ Basic Indexing (1D, 2D, nD)

Individual elements are accessed using their position index, starting from 0. In multi-dimensional arrays, indices are given for each axis.

```
arr1d = np.array([10, 20, 30, 40])
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
arr3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

print(arr1d[2])
print(arr2d[1, 2])
print(arr3d[1, 0, 1])
```

```
30
6
6
```

✓ Slicing (: operator)

Slicing selects a range of elements using start:stop:step notation. The stop index is excluded.

```
arr = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
print(arr[2:6])
```

```
[2 3 4 5]
```

✓ Negative Indexing

Negative indices count from the end of the array, with -1 referring to the last element.

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[-1])
print(arr[-3:])
```

```
50
[30 40 50]
```

✓ Step Slicing

A step value can be added to a slice to skip elements.

```
arr = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
print(arr[:2])
print(arr[::-1])
```

```
[0 2 4 6 8]
[9 8 7 6 5 4 3 2 1 0]
```

✓ Fancy Indexing

Fancy indexing selects elements using a list or array of indices, allowing arbitrary and non-contiguous selections.

```
arr = np.array([10, 20, 30, 40, 50])
indices = [0, 2, 4]
print(arr[indices])
```

```
[10 30 50]
```

✓ Boolean Indexing

Boolean indexing selects elements based on a condition, returning only the elements where the condition is True.

```
arr = np.array([10, 15, 20, 25, 30])
print(arr[arr > 15])
```

```
[20 25 30]
```

✓ Indexing with np.where()

np.where() returns the indices where a condition is True, or can be used to choose values from two arrays based on a condition.

```
arr = np.array([10, 15, 20, 25, 30])
print(np.where(arr > 15))
print(np.where(arr > 15, arr, 0))
```

```
(array([2, 3, 4]),)
[ 0  0 20 25 30]
```

✓ 7. Array Manipulation

np.concatenate(), np.vstack(), np.hstack(), np.dstack()

- np.concatenate() joins arrays along an existing axis.
- np.vstack() stacks arrays vertically (row-wise).
- np.hstack() stacks arrays horizontally (column-wise).
- np.dstack() stacks arrays along a third (depth) axis.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.concatenate([a, b]))
print(np.vstack([a, b]))
print(np.hstack([a, b]))
print(np.dstack([a, b]))
```

```
[1 2 3 4 5 6]
[[1 2 3]
 [4 5 6]]
[1 2 3 4 5 6]
[[[1 4]
  [2 5]
  [3 6]]]
```

np.split(), np.array_split()

np.split() divides an array into equal parts and raises an error if the split is uneven. np.array_split() allows uneven splits.

```
arr = np.arange(9)
print(np.split(arr, 3))

arr2 = np.arange(10)
print(np.array_split(arr2, 3))
```

```
[array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8])]
[array([0, 1, 2, 3]), array([4, 5, 6]), array([7, 8, 9])]
```

np.insert(), np.append(), np.delete()

- np.insert() inserts values at a given index.
- np.append() adds values to the end of an array.
- np.delete() removes elements at a given index.

```
arr = np.array([1, 2, 3, 4])

print(np.insert(arr, 2, 99))
print(np.append(arr, 5))
print(np.delete(arr, 1))
```

```
[ 1  2 99  3  4]
[1 2 3 4 5]
[1 3 4]
```

Stacking vs Concatenation

Concatenation joins arrays along an existing axis without adding a new dimension. Stacking (vstack, hstack, dstack, stack) joins arrays along a new axis, which can increase the number of dimensions.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.concatenate([a, b]).shape)
print(np.stack([a, b]).shape)
```

```
(6,)
(2, 3)
```

Copy vs View

A view shares memory with the original array, so changes to one affect the other. A copy creates an independent array in new memory. 

`.view()` creates a view, and `.copy()` creates a copy.

```
arr = np.array([1, 2, 3, 4])

view_arr = arr.view()
copy_arr = arr.copy()

arr[0] = 100

print("Original:", arr)
print("View (changed too):", view_arr)
print("Copy (unchanged):", copy_arr)
```

```
Original: [100  2  3  4]
View (changed too): [100  2  3  4]
Copy (unchanged): [1 2 3 4]
```

8. Useful Utility Functions

np.unique()

Returns the sorted unique elements of an array.

```
arr = np.array([1, 2, 2, 3, 3, 3, 4])
print(np.unique(arr))
```

```
[1 2 3 4]
```

np.sort(), np.argsort()

`np.sort()` returns a sorted copy of the array. `np.argsort()` returns the indices that would sort the array.

```
arr = np.array([3, 1, 4, 1, 5, 9])
print(np.sort(arr))
print(np.argsort(arr))
```

```
[1 1 3 4 5 9]
[1 3 0 2 4 5]
```

np.where(), np.nonzero()

`np.where()` finds indices matching a condition (already shown above). `np.nonzero()` returns the indices of all non-zero elements.

```
arr = np.array([0, 1, 0, 3, 0, 5])
print(np.nonzero(arr))
```

```
(array([1, 3, 5]),)
```

np.any(), np.all()

`np.any()` returns True if at least one element satisfies a condition. `np.all()` returns True only if all elements satisfy a condition.

```
arr = np.array([1, 2, 0, 4])
print(np.any(arr > 3))
print(np.all(arr > 0))
```

```
True
False
```

np.isnan(), np.isinf(), np.isfinite()

`np.isnan()` checks for NaN (Not a Number) values. `np.isinf()` checks for infinite values. `np.isfinite()` checks for values that are neither NaN nor infinite.

```
arr = np.array([1, np.nan, np.inf, -np.inf, 5])

print(np.isnan(arr))
print(np.isinf(arr))
print(np.isfinite(arr))

[False  True  False  False  False]
[False  False  True   True  False]
[ True  False  False  False  True]
```

✓ np.clip()

Limits the values in an array to a given minimum and maximum, clipping values outside that range.

```
arr = np.array([1, 5, 10, 15, 20])
print(np.clip(arr, 5, 15))

[ 5  5 10 15 15]
```

✓ Handling NaN

`np.nan` represents a missing or undefined value. `np.isnan()` detects NaN values. `np.nanmean()`, `np.nanstd()`, `np.nansum()` compute mean, standard deviation, and sum while ignoring NaN values.

```
arr = np.array([1, 2, np.nan, 4, 5])

print(np.isnan(arr))
print(np.nanmean(arr))
print(np.nanstd(arr))
print(np.nansum(arr))

[False False  True  False  False]
3.0
1.5811388300841898
12.0
```

✓ 9. Mini NumPy Case Study

This case study works with a small structured dataset of product prices and quantities. It covers creating arrays, cleaning data, performing operations, and generating a summary.

✓ Step 1: Create Arrays

Create arrays representing product names, prices, and quantities. The prices array intentionally contains a missing value (NaN) to simulate real data.

```
products = np.array(["Pen", "Notebook", "Bag", "Bottle", "Pencil"])
prices = np.array([10.0, 45.5, np.nan, 120.0, 5.0])
quantities = np.array([100, 60, 20, 15, 200])

print("Products:", products)
print("Prices:", prices)
print("Quantities:", quantities)

Products: ['Pen' 'Notebook' 'Bag' 'Bottle' 'Pencil']
Prices: [ 10.  45.5  nan 120.   5. ]
Quantities: [100  60  20  15 200]
```

✓ Step 2: Clean Data

Replace the missing price with the average of the known prices using `np.nanmean()`, so no valid calculations are lost.

```
mean_price = np.nanmean(prices)
clean_prices = np.where(np.isnan(prices), mean_price, prices)

print("Cleaned prices:", clean_prices)
```

```
Cleaned prices: [ 10.    45.5   45.125 120.    5.   ]
```

▼ Step 3: Perform Operations

Calculate the total value of stock for each product (price times quantity), and find products with high quantity using boolean indexing.

```
stock_value = clean_prices * quantities
print("Stock value per product:", stock_value)

high_stock = products[quantities > 50]
print("Products with quantity above 50:", high_stock)
```

```
Stock value per product: [1000.  2730.   902.5 1800.  1000. ]
Products with quantity above 50: ['Pen' 'Notebook' 'Pencil']
```

▼ Step 4: Generate Summary

Use aggregate and statistical functions to summarize the dataset.

```
print("Total inventory value:", np.sum(stock_value))
print("Average price:", np.mean(clean_prices))
print("Maximum stock value product:", products[np.argmax(stock_value)])
print("Minimum stock value product:", products[np.argmin(stock_value)])
print("Sorted products by stock value:", products[np.argsort(stock_value)])
```

```
Total inventory value: 7432.5
Average price: 45.125
Maximum stock value product: Notebook
Minimum stock value product: Bag
Sorted products by stock value: ['Bag' 'Pen' 'Pencil' 'Bottle' 'Notebook']
```